Architetture parallele per una formulazione continua della Particle Swarm Optimization

Giovanni Giuseppe Iacuzzo[1,*]

[1]Dipartimento di Ingegneria e Architettura, Università degli Studi di Enna "Kore", Enna, Italia

[*]Autore corrispondente: giovanni.iacuzzo@unikore.it

Sommario

La Particle Swarm Optimization continua (CPSO) descrive l'evoluzione di una particella mediante una dinamica differenziale propagata in forma analitica. La sua parallelizzazione richiede di specificare quando vengono osservate le traiettorie e quando le informazioni personali e sociali diventano disponibili. Questo lavoro definisce e implementa cinque modalità di esecuzione: un riferimento seriale sincrono, master–slave, particle-parallel sincrona, asincrona event-driven e a isole. Le prime due architetture parallele sincrone sono progettate per preservare la semantica del riferimento, mentre le ultime due introducono differenti dinamiche di aggiornamento e scambio dell'informazione. Sono considerate tre topologie sociali (globale, anello e griglia toroidale di von Neumann). Il protocollo interno analizza sei funzioni in dimensione 30 e 100 e, separatamente, il costo dell'esecutore mediante un carico sintetico. Il software include inoltre driver seriali per COCO BBOB, COCO BBOB-LargeScale e CEC 2013 LSGO, confrontati a budget FE fissato con sei baseline PSO discrete (gbest classica, inerzia, costrizione, lbest ad anello, CLPSO e SL-PSO). Questi benchmark esterni costituiscono un livello di valutazione distinto e non misurano lo scaling delle architetture parallele. Il manoscritto presenta la specificazione operativa e i protocolli; le campagne esterne e interne definitive e la verifica integrale di fedeltà al testo sorgente sono ancora da completare.

Parole chiave: Particle Swarm Optimization; PSO a tempo continuo; parallelizzazione; algoritmi sincroni e asincroni; modelli a isole; equivalenza algoritmica; analisi sperimentale.

1 Introduzione

La Particle Swarm Optimization (PSO) è una metaeuristica basata su popolazione nella quale le particelle aggiornano il proprio stato usando informazioni individuali e sociali [1]. La semplicità della formulazione e la possibilità di valutare separatamente le funzioni obiettivo delle particelle hanno favorito lo sviluppo di numerose implementazioni parallele. La letteratura distingue, tra le altre, strategie master–slave, modelli a isole e organizzazioni a grana fine; le implementazioni possono inoltre differire per sincronizzazione e piattaforma di esecuzione [2–4]. Queste scelte non sono tutte equivalenti: alcune distribuiscono il lavoro mantenendo invariata la sequenza logica degli aggiornamenti, mentre altre cambiano le informazioni disponibili alle particelle e quindi definiscono una dinamica di ricerca differente.

La distinzione è particolarmente importante quando si considera una formulazione a tempo continuo. In questo lavoro prendiamo come riferimento la formulazione continua studiata da Orlando and Ricciardello [5], nella quale il moto della particella è descritto attraverso un problema di Cauchy per un'equazione differenziale ordinaria del secondo ordine e viene utilizzata una soluzione analitica. L'aggettivo *continuo* si riferisce al parametro temporale della dinamica, non alla natura dello spazio decisionale. Esistono altre formulazioni dinamiche e stocastiche della PSO, con ipotesi e proprietà differenti [6–8]; pertanto, i risultati ottenuti per una variante non si trasferiscono automaticamente alle altre. Nel seguito usiamo la sigla CPSO per riferirci al modello continuo associato a Orlando and Ricciardello [5], senza implicare che l'implementazione qui studiata ne costituisca già una riproduzione certificata in ogni dettaglio. La verifica equazione-per-equazione rispetto al testo integrale e la risoluzione delle convenzioni operative ancora aperte sono prerequisiti espliciti per la validazione finale del modello.

La parallelizzazione di un modello a tempo continuo richiede una definizione operativa degli istanti di osservazione. Una traiettoria analitica, da sola, non specifica quando valutare la funzione obiettivo, quando aggiornare i migliori personali e sociali, né quali informazioni debbano essere visibili agli altri agenti. In questo lavoro il moto viene quindi suddiviso in intervalli a coefficienti congelati: all'inizio dell'intervallo sono fissati i coefficienti casuali e l'informazione sociale, la soluzione analitica propaga lo stato per un intervallo di lunghezza prescritta e la funzione obiettivo viene valutata all'estremo. Questa discretizzazione degli eventi di osservazione è una scelta algoritmica esplicita; non va confusa con un errore di discretizzazione di un integratore numerico, perché il propagatore locale è valutato in forma chiusa.

Studiamo cinque modalità di esecuzione implementate nel medesimo motore C++: (i) riferimento seriale sincrono; (ii) master–slave, che parallelizza le valutazioni obiettivo mantenendo seriali propagazione e commit; (iii) particle-parallel sincrona, che parallelizza anche il lavoro per particella ma mantiene congelato lo snapshot sociale e ordina deterministicamente il commit; (iv) asincrona event-driven, nella quale ogni evento completato rende immediatamente visibile l'aggiornamento del migliore personale; e (v) modello a isole, nel quale sottopopolazioni evolvono separatamente e possono scambiare emigranti a intervalli prefissati. Le prime due architetture parallele sincrone sono progettate per preservare la semantica del riferimento seriale; le modalità asincrona e a isole modificano invece la dinamica e non sono interpretate come semplici accelerazioni dello stesso algoritmo. Tutte le architetture sono implementate su memoria condivisa con thread CPU; non sono oggetto della versione corrente backend MPI, distribuiti o GPU.

La topologia sociale è un fattore distinto dall'esecutore. Consideriamo informazione globale, vicinato ad anello periodico e griglia toroidale di von Neumann, combinazioni che consentono di isolare l'effetto della struttura di comunicazione logica rispetto alla modalità di esecuzione. La topologia ad anello e le strutture locali sono consolidate nella letteratura PSO e possono produrre comportamenti diversi dal modello globale [9]. Nel nostro studio, la selezione del leader sociale usa uno snapshot coerente della memoria dei migliori personali all'inizio di ogni generazione sincrona; nelle modalità asincrona e a isole valgono invece le regole di pubblicazione e migrazione definite dalle rispettive architetture.

L'obiettivo è costruire e verificare un quadro sperimentale che separi la conservazione della traiettoria dall'alterazione della dinamica. Le domande di ricerca sono:

- **RQ1.** Quali condizioni operative rendono ben definita la semantica sincrona del modello continuo e consentono a master–slave e particle-parallel di riprodurre esattamente il riferimento seriale?
- **RQ2.** A parità di popolazione totale e budget di valutazioni, come topologia sociale, aggiornamento event-driven e decomposizione a isole influenzano la qualità della soluzione?
- **RQ3.** Per le architetture sincrone che preservano la traiettoria, in quali regimi di dimensione, popolazione e costo della valutazione l'esecuzione parallela riduce il tempo reale, e quale efficienza parallela raggiunge?

Il protocollo di qualità utilizza sei funzioni analitiche deterministiche (Sphere, Rastrigin, Rosenbrock, Ackley, Griewank e Salomon), nelle dimensioni 30 e 100, con popolazione totale pari a 64 particelle e 500 aggiornamenti. Le campagne confrontano topologie, configurazioni asincrone e configurazioni a isole usando semi appaiati e lo stesso budget di valutazioni per ciascuna istanza funzione-dimensione. Questo insieme è un banco di prova interno per l'analisi delle architetture, non una suite esterna standardizzata: non viene presentato come sostituto di COCO/BBOB o CEC e non consente, da solo, affermazioni di superiorità generale o state of the art.

La valutazione temporale è separata dalla qualità di ottimizzazione. Per studiare il rapporto tra costo delle valutazioni e overhead del parallelismo utilizziamo esclusivamente la funzione sintetica `sphere_work`, che aggiunge un carico aritmetico controllato alla funzione Sphere. I suoi risultati sono utilizzati soltanto per analizzare il comportamento dell'esecutore e il punto di pareggio del parallelismo; non sono aggregati ai risultati delle sei funzioni di ottimizzazione e non vengono interpretati come prestazioni su un benchmark standard. Il tempo reale, lo speedup e l'efficienza sono riportati soltanto per master–slave e particle-parallel dopo aver verificato l'equivalenza esatta rispetto al riferimento seriale, inclusi valore e posizione migliori, cronologia del best-so-far, asse delle valutazioni e numero totale di valutazioni.

L'analisi statistica mantiene l'appaiamento dei semi all'interno di ciascuna coppia funzione-dimensione. Le sei funzioni sono trattate come cluster di livello superiore; le due dimensioni sono misure ripetute annidate nella stessa famiglia e i semi sono repliche annidate nella coppia funzione-dimensione. Le inferenze confermative sono pertanto condotte su sei riassunti di famiglia, mentre i dodici blocchi funzione-dimensione sono mantenuti per la descrizione e l'analisi dimensione-specifica. Il numero ridotto di famiglie indipendenti limita la potenza inferenziale e impone di interpretare intervalli, dimensioni dell'effetto e valori di probabilità con cautela.

I contributi del manoscritto, nella versione corrente, sono circoscritti a:

- una specificazione operativa delle semantiche di aggiornamento sincrone, asincrone e a isole per il modello continuo considerato, con una distinzione esplicita tra equivalenza algoritmica e cambiamento della dinamica;
- un'implementazione CPU a memoria condivisa di cinque modalità di esecuzione e tre topologie sociali, con test di equivalenza esatta per le due architetture sincrone parallele;
- un protocollo a budget di valutazioni controllato per studiare topologie, asincronia e isole su sei funzioni analitiche, insieme a un protocollo separato di strong scaling basato su carico sintetico;
- una procedura di analisi appaiata che tratta la funzione obiettivo come cluster di livello superiore e mantiene distinti risultati di qualità e misure di tempo reale;
- un livello software separato per l'esecuzione controllata su COCO BBOB, BBOB-LargeScale e CEC 2013 LSGO, con sei baseline PSO a tempo discreto in spazio continuo esplicitamente parametrizzate e budget FE abbinato; questa estensione definisce il protocollo di valutazione esterna ma non anticipa risultati né costituisce un backend parallelo.

Questa versione del manoscritto documenta modello, implementazione e protocollo; i risultati delle campagne definitive non sono ancora inclusi e non vengono anticipati. La repository include ora un livello di valutazione esterna separato, con driver COCO BBOB e BBOB-LargeScale, CEC 2013 LSGO e sei baseline PSO a tempo discreto in spazio continuo. Tali driver usano un callback seriale e servono al confronto di qualità a budget FE fissato: non implementano un nuovo backend parallelo e non sono utilizzati per inferire speedup delle architetture native. Le campagne esterne devono ancora essere eseguite con i pacchetti ufficiali e versioni ambientali congelate. Backend a memoria distribuita/GPU e decomposizione cooperativa delle variabili restano sviluppi distinti.

2 Modello matematico e semantica operativa

2.1 Stato della particella e dinamica a coefficienti congelati

Sia N il numero di particelle e D la dimensione dello spazio decisionale. Per la particella i allo step osservativo k indichiamo con $x_i^k, v_i^k \in \mathbb{R}^D$ posizione e velocità, con p_i^k la posizione del migliore personale e con s_i^k il leader sociale determinato dalla topologia scelta. Il valore obiettivo è $J(x)$. Il modello implementato aggiorna la dinamica su intervalli di durata Δt nei quali coefficienti casuali, memoria personale, informazione sociale e parametro di inerzia restano congelati.

Per ogni particella e intervallo vengono estratti due scalari $r_{1,i}^k, r_{2,i}^k \in [0, 1)$. Poniamo

$$a_i^k = c_1 r_{1,i}^k, \qquad b_i^k = c_2 r_{2,i}^k, \qquad q_i^k = a_i^k + b_i^k, \quad (1)$$

e definiamo $F_i^k = a_i^k p_i^k + b_i^k s_i^k$. Per ogni coordinata j la traiettoria locale soddisfa

$$\ddot{x}_{i,j}(t) + (1 - \mu_k)\dot{x}_{i,j}(t) + q_i^k x_{i,j}(t) = F_{i,j}^k, \quad t \in [0, \Delta t], \quad (2)$$

con condizioni iniziali $x_{i,j}(0) = x_{i,j}^k$ e $\dot{x}_{i,j}(0) = v_{i,j}^k$. Gli scalari casuali sono condivisi da tutte le coordinate della stessa particella nello stesso intervallo: questa è una convenzione esplicita dell'implementazione corrente e resta soggetta al confronto con il testo integrale della formulazione sorgente.

Quando $q_i^k > 0$, l'equilibrio della dinamica congelata è $x_{i,j}^{\mathrm{eq}} = F_{i,j}^k / q_i^k$. La parte omogenea ha equazione caratteristica

$$\lambda^2 + (1 - \mu_k)\lambda + q_i^k = 0, \qquad \alpha_k = \frac{1 - \mu_k}{2}. \quad (3)$$

Il propagatore $\Phi_{\Delta t}$ è valutato analiticamente nei regimi sottosmorzato, criticamente smorzato e sovrasmorzato; il caso $q_i^k = 0$ è trattato separatamente. Non viene impiegato un integratore temporale numerico per approssimare la traiettoria all'interno dell'intervallo. La funzione obiettivo è valutata soltanto all'estremo $t = \Delta t$.

Per una run con K aggiornamenti, il parametro di inerzia varia secondo

$$\mu_k = \mu_{\max} - (\mu_{\max} - \mu_{\min}) \frac{k}{K - 1}, \qquad k = 0, \ldots, K - 1, \quad (4)$$

quando $K > 1$; per $K \le 1$ viene utilizzato $\mu_{\max}$. Le posizioni iniziali sono campionate uniformemente e indipendentemente per coordinata nell'intervallo ammissibile. Le velocità iniziali seguono

$$v_{i,j}^0 \sim U\left[-\rho(u_j - \ell_j), \rho(u_j - \ell_j)\right], \quad (5)$$

con ρ pari al parametro `initial_velocity_fraction`.

2.2 Vincoli, memoria e topologia sociale

La soluzione analitica viene seguita da una mappa discreta di gestione dei vincoli. Se una coordinata oltrepassa il limite inferiore ℓ_j, la posizione viene proiettata su ℓ_j e una velocità uscente viene sostituita con $-e v_{i,j}$; il caso del limite superiore u_j è simmetrico. Il coefficiente $e \in [0, 1]$ è il parametro di restituzione. Questa mappa è una regola algoritmica applicata agli estremi degli intervalli, non una condizione al contorno dell'ODE. Non vengono rilevati attraversamenti multipli all'interno dello stesso intervallo: la posizione intermedia della traiettoria analitica non viene monitorata.

Il miglior personale è aggiornato soltanto in presenza di miglioramento stretto,

$$p_i^{k+1} = \begin{cases} x_i^{k+1}, & J(x_i^{k+1}) < J(p_i^k), \\ p_i^k, & \text{altrimenti.} \end{cases} \quad (6)$$

I pareggi nella selezione di un leader sociale sono risolti scegliendo l'indice di particella minore. Le tre topologie implementate sono: (i) globale, nella quale tutte le particelle usano il migliore personale globale; (ii) anello periodico di raggio uno; (iii) griglia toroidale di von Neumann, con sé stessa e i quattro vicini ortogonali. La griglia richiede $RC = N$ e dimensioni almeno pari a due; non è attualmente ammessa come topologia interna del modello a isole.

2.3 Semantica sincrona e proprietà di equivalenza

In una generazione sincrona, tutti i leader sociali vengono selezionati dallo stesso snapshot $P^k = (p_1^k, \ldots, p_N^k)$. I propagatori e le valutazioni obiettivo possono essere eseguiti in qualunque ordine perché ciascun risultato è scritto nello slot associato alla particella. Gli stati personali e globali sono poi aggiornati in ordine deterministico degli indici. Master–slave parallelizza soltanto le valutazioni obiettivo; particle-parallel parallelizza anche inizializzazione e propagazione delle particelle, mantenendo lo snapshot sociale congelato e il commit ordinato.

Proposizione 2.1 (Invarianza rispetto alla schedulazione sincrona)**.** *Si considerino una configurazione e un seme fissati. Si assumano: (i) funzione obiettivo deterministica e priva di effetti collaterali, thread-safe quando invocata in parallelo; (ii) stato pseudocasuale assegnato a ciascuna particella e consumato esclusivamente da essa; (iii) selezione dei leader dallo stesso snapshot pre-generazione; (iv) identiche mappe di propagazione, gestione dei vincoli, regole di miglioramento e tie-breaking; (v) commit deterministico in ordine di indice. Allora le esecuzioni seriale, master–slave e particle-parallel producono la stessa sequenza di stati osservati, degli incumbent e dei valori best-so-far, a meno delle differenze dovute a un diverso ambiente floating-point o a una diversa implementazione della funzione obiettivo.*

Dimostrazione. All'inizializzazione, ogni particella usa lo stream pseudocasuale associato al proprio indice e genera quindi lo stesso stato iniziale nelle tre esecuzioni. Supponiamo per induzione che, all'inizio della generazione k, posizioni, velocità, migliori personali e stati pseudocasuali coincidano. La selezione sociale dipende soltanto dallo snapshot comune P^k e dalla stessa topologia, perciò ogni particella riceve il medesimo attrattore. Lo stream privato della particella produce la stessa coppia $(r_{1,i}^k, r_{2,i}^k)$, indipendentemente dal thread che ne esegue il lavoro. La stessa mappa analitica e la stessa mappa dei vincoli producono dunque la medesima posizione candidata; la deterministica funzione obiettivo restituisce lo stesso valore. Le computazioni per particelle diverse non modificano lo stato condiviso prima del commit e i risultati vengono scritti in posizioni indicizzate. Il commit, eseguito con il medesimo

ordine e le medesime regole di confronto, genera gli stessi migliori personali e lo stesso incumbent globale. Gli stream pseudocasuali hanno consumato lo stesso numero di estrazioni e restano allineati. Lo stato completo all'inizio di $k+1$ coincide, chiudendo il passo induttivo. □

La proposizione esplicita il contratto algoritmico che motiva il test di equivalenza implementativo. In aritmetica floating-point fissata e usando la stessa libreria di obiettivo, il progetto verifica l'uguaglianza esatta dei risultati osservati. Non ne segue equivalenza per funzioni obiettivo non deterministiche, non thread-safe, o per esecuzioni su compilatori e architetture floating-point differenti.

2.4 Semantica asincrona e a isole

Nell'architettura event-driven, dopo l'inizializzazione viene inserito in una coda un evento $(i, 0)$ per ciascuna particella. Un evento (i, k) legge sotto mutex il migliore personale della particella e l'attrattore sociale corrente, propaga e valuta il candidato, pubblica immediatamente un eventuale miglioramento e reinserisce $(i, k + 1)$ nella coda, se richiesto. Una particella non ha due eventi simultaneamente in volo: il successivo evento viene inserito soltanto dopo il completamento del precedente. Non esiste tuttavia una barriera globale di generazione; con più worker, l'ordine di completamento e quindi gli snapshot sociali osservati può dipendere dalla schedulazione.

Nel modello a isole la popolazione totale N è divisa in M sottopopolazioni uguali. Ogni isola esegue una generazione sincrona indipendente. Quando è previsto un evento di migrazione, gli incumbent di tutte le isole vengono copiati in uno snapshot immutabile; solo dopo aver costruito tale snapshot sono applicate le sostituzioni. La migrazione ad anello invia l'incumbent dell'isola precedente, mentre quella fully-connected seleziona il migliore incumbent tra le altre isole. L'immigrato sostituisce il peggiore migliore personale solo se strettamente migliore e la velocità del destinatario viene azzerata. Un emigrante possiede già il proprio valore obiettivo, pertanto la migrazione non aumenta il numero di valutazioni.

Tutte le modalità mantengono il budget complessivo

$$N_{\mathrm{FE}} = N(K + 1), \tag{7}$$

comprensivo della valutazione iniziale di ogni particella. Le storie sincrone sono registrate a ogni generazione; quella asincrona dopo ogni evento di aggiornamento e include esplicitamente l'asse delle valutazioni. Per costruzione, le modalità asincrone e a isole sono algoritmi con dinamiche differenti e il loro rapporto di tempo con il riferimento non viene denominato speedup parallelo.

Limite di fedeltà alla fonte. Le equazioni e le convenzioni sopra riportate descrivono con precisione l'implementazione sottoposta a test. La verifica della corrispondenza puntuale con la formulazione pubblicata da Orlando and Ricciardello [5] resta aperta finché non sarà disponibile il testo integrale; in particolare, il carattere scalare o vettoriale dei coefficienti casuali e le convenzioni su inerzia, condizioni iniziali e vincoli devono essere confrontati equazione per equazione. La proposizione 2.1 dimostra l'equivalenza tra esecutori rispetto a questo modello implementato, non la sua equivalenza con un modello sorgente non ancora verificato.

3 Protocollo sperimentale

3.1 Disegno della campagna

La campagna predefinita è articolata in quattro stadi, separando gli effetti sulla dinamica di ricerca dalla misurazione dello scaling computazionale. Per gli stadi di qualità si considerano Sphere, Rastrigin, Rosenbrock, Ackley, Griewank e Salomon, in dimensione $D \in \{30, 100\}$, con popolazione totale $N = 64$, $K = 500$ aggiornamenti e 30 semi appaiati. Ogni run utilizza quindi $N(K + 1) = 32\,064$ valutazioni obiettivo.

Il primo stadio isola la topologia sociale in esecuzione seriale. Il secondo confronta la semantica event-driven a uno e quattro worker con i riferimenti seriali corrispondenti. Il terzo studia il numero di isole, la topologia interna e le politiche di migrazione ad anello o fully-connected mantenendo costante N e il budget FE. Le configurazioni esatte e le combinazioni sono registrate in `configs/campaigns/definitive_architecture_campaign.json`.

3.2 Misura dello scaling

Il quarto stadio usa esclusivamente 'sphere_work', una funzione sintetica basata su Sphere con una ricorrenza aritmetica controllabile. Sono considerati $D \in \{30, 100\}$, $N \in \{64, 128\}$, $K = 80$, carichi di 0, 512 e 4 096 iterazioni per valutazione e 10 semi appaiati. Per ogni backend sincrono si misurano 1, 2, 4 e 8 worker. Il carico sintetico non è incluso nell'analisi di qualità e non viene presentato come benchmark di ottimizzazione.

Lo speedup e l'efficienza sono definiti come

$$S(p) = \frac{T_{\mathrm{serial}}}{T_p}, \qquad E(p) = \frac{S(p)}{p}, \tag{8}$$

ma vengono riportati soltanto per master–slave e particle-parallel dopo il superamento del controllo di equivalenza esatta del paragrafo 2. Le modalità asincrona e a isole non sono confrontate in termini di speedup contro il seriale, poiché non preservano la stessa traiettoria.

3.3 Unità statistiche e inferenza

I semi sono appaiati all'interno di ciascuna coppia funzione-dimensione. Le sei funzioni costituiscono i cluster di livello superiore; le dimensioni sono misure ripetute annidate nella famiglia di funzione e i semi sono repliche appaiate annidate nella cella funzione-dimensione. Per un confronto tra metodo A e riferimento B, si calcola prima la mediana delle differenze appaiate $\log_{10}(J_A) - \log_{10}(J_B)$ tra semi in ogni cella funzione-dimensione; quindi si riducono le due dimensioni alla mediana di famiglia. Il numero di unità indipendenti per l'inferenza confermativa è pertanto sei, non dodici e non il numero complessivo dei semi.

Le differenze a livello di famiglia sono sottoposte a test signed-rank bilaterale con enumerazione esatta delle configurazioni di segno; i valori di probabilità sono corretti con la procedura di Holm all'interno della famiglia di confronti pre-specificata. L'analisi omnibus di Friedman è basata sui ranghi dei metodi all'interno di ogni famiglia e usa 20 000 permutazioni dei nomi dei metodi entro famiglia per calibrare il valore di probabilità. I valori asintotici chi-quadro e Iman–Davenport sono conservati soltanto come diagnostica secondaria. Gli intervalli al 95% sono ottenuti con bootstrap

Tabella 1: Struttura della campagna predefinita. Gli stadi 1–3 misurano qualità con budget di valutazioni fissato; lo stadio 4 misura esclusivamente il tempo dell'esecutore su un carico sintetico.

Stadio	Confronto principale	Run	Valutazioni obiettivo
1. Topologie	Serial-global, serial-ring, serial-grid	1 080	34 629 120
2. Asincronia	Tre topologie; asincrono con 1 e 4 worker; riferimenti seriali abbinati	3 240	103 887 360
3. Isole	2, 4 e 8 isole; topologia locale e politiche di migrazione	3 960	126 973 440
4. Strong scaling	Serial, master–slave e particle-parallel; 1, 2, 4 e 8 worker	1 080	8 398 080
Totale		9 360	273 888 000

annidato: ricampionamento delle famiglie, delle dimensioni dentro ciascuna famiglia e dei semi appaiati dentro ciascuna cella.

Il numero ridotto di famiglie limita la potenza e la generalizzabilità statistica. I conteggi di vittorie, sconfitte e pareggi a livello run e le statistiche separate per funzione-dimensione sono riportati a fini descrittivi, ma non vengono trattati come osservazioni indipendenti. Il target di successo è 10^{-8} per le funzioni con ottimo globale nullo; le run senza successo sono censurate al budget FE nel calcolo dell'ERT. In assenza di successi l'ERT è infinito e deve essere accompagnato dal numero di successi.

3.4 Riproducibilità e stato dei risultati

Ogni run è identificata da funzione, dimensione, seme, metodo e hash della configurazione; il manifest conserva inoltre hash del codice, versione/ABI nativa, compilatore, flag di compilazione, CPU, sistema, versioni software e variabili d'ambiente relative al threading. I record sono salvati in JSON Lines append-only e sincronizzati su disco dopo il completamento di ciascuna run. Le campagne sono riprendibili: in fase di resume può essere eliminata soltanto una riga finale interrotta, mentre la corruzione di record completi determina un errore.

La campagna definitiva è definita e validata nel file di configurazione, ma non è stata eseguita integralmente nella versione corrente. I risultati dello smoke test servono soltanto a verificare orchestrazione, contabilità FE, controlli di equivalenza, analisi e generazione delle figure; non sono utilizzati per sostenere conclusioni scientifiche.

3.5 Benchmark esterni e baseline PSO discrete

La valutazione interna delle architetture non sostituisce benchmark esterni standardizzati. Per questo motivo il software include un livello distinto di valutazione della qualità della ricerca su tre famiglie di problemi: COCO `bbob`, COCO `bbob-largescale` e CEC 2013 LSGO [10–12]. I benchmark sono richiamati tramite adapter esterni e non vengono ricostruiti nel codice del progetto. Gli adapter non modificano le architetture del motore: l'obiettivo esterno è invocato attraverso un callback seriale, perché non si assume che le implementazioni Python/Cython dei benchmark siano thread-safe.

Per le due suite COCO, ogni metodo riceve un oggetto `Suite` e un `Observer` nuovi, così da separare contatori e file di log. Il driver verifica la corrispondenza tra il numero di chiamate registrate dall'oggetto COCO e il contatore FE dell'ottimizzatore; i dati observer sono mantenuti per il post-processing standard con `cocopp`. La suite BBOB standard usa le dimensioni $2, 3, 5, 10, 20, 40$, mentre BBOB-LargeScale usa $20, 40, 80, 160, 320, 640$ [11]. Per entrambe le suite il progetto seleziona un budget di $10^4 D$ valutazioni per problema e metodo. Tale budget è una scelta sperimentale del progetto e non viene presentato come requisito normativo di COCO.

Per CEC 2013 LSGO vengono utilizzate le 15 funzioni a 1000 variabili, con 25 semi indipendenti e budget massimo di $3 \cdot 10^6$ valutazioni per run. Sono registrati i valori best-so-far a $1.2 \cdot 10^5$, $6 \cdot 10^5$ e $3 \cdot 10^6$ FE, come previsto dal rapporto tecnico della competizione [12]. I limiti di ricerca sono letti dai metadati dell'adapter; la mappa di vincolo comune del presente studio applica il clipping alla frontiera e la restituzione della velocità con coefficiente 0.5. Questa è una scelta operativa uniforme dei metodi, non una proprietà intrinseca delle funzioni CEC.

I metodi confrontati sono la CPSO continua nativa e sei baseline PSO a tempo discreto in spazio continuo implementate separatamente in NumPy. La prima baseline è una PSO global-best classica, con $c_1 = c_2 = 2$ e velocità inerziale non pesata [1]. La seconda utilizza un coefficiente di inerzia lineare da 0.9 a 0.4, con $c_1 = c_2 = 2.05$ [13]. La terza applica il fattore di costrizione

$$\chi = \frac{2}{\varphi - 2 + \sqrt{\varphi^2 - 4\varphi}}, \qquad \varphi = c_1 + c_2 > 4, \tag{9}$$

con $c_1 = c_2 = 2.05$ [6]. La quarta utilizza la stessa inerzia lineare della seconda ma sostituisce l'informazione globale con il miglior personale in un vicinato ad anello periodico di raggio uno [9]. Le prime tre baseline usano l'informazione global-best. La quinta baseline è il Comprehensive Learning PSO (CLPSO) [14]: ogni coordinata apprende dal personal best di un esemplare selezionato mediante torneo, con probabilità specifica della particella $P_{c,i} = 0.05 + 0.45[\exp(10i/(N-1)) - 1]/[\exp(10) - 1]$, $i = 0, \ldots, N-1$. L'aggiornamento utilizza $c = 1.49445$, inerzia lineare da 0.9 a 0.4, limite di velocità pari a 0.2 della larghezza del dominio e rinnovo degli esemplari dopo sette generazioni senza miglioramento del personal best. Il ciclo CLPSO è implementato per generazioni con uno snapshot congelato di posizioni e personal best; ciò evita che l'ordine di valutazione induca una dinamica asincrona non dichiarata. Per garantire un trattamento comune dei vincoli viene adottata la medesima mappa clamp–restitution usata dagli altri metodi, scelta che viene dichiarata come convenzione del presente protocollo.

La sesta baseline è SL-PSO (Social Learning PSO) [15], concepita per problemi scalabili. Il gruppo viene ordinato dal peggiore al migliore; ogni particella, eccetto la migliore, può apprendere da dimostratori con fitness migliore. Con indice di rango $i = 0, \ldots, N-2$, dimensione D, $M = 100$, $\alpha = 0.5$ e $\beta = 0.01$, la probabilità di apprendimento e il fattore di influenza sociale sono

Tabella 2: Protocollo predefinito per le suite esterne. I budget BBOB sono una scelta del presente studio; i parametri CEC riportati seguono il protocollo pubblicato. Le configurazioni sono predefinite ma le campagne non sono ancora state eseguite integralmente.

Suite	Problemi	Dimensioni	Istanze/semi	Budget per run
COCO BBOB	24 funzioni	$2, 3, 5, 10, 20, 40$	15 istanze	$10^4 D$ FE
COCO BBOB-LargeScale	24 funzioni	$20, 40, 80, 160, 320, 640$	15 istanze	$10^4 D$ FE
CEC 2013 LSGO	15 funzioni	$D = 1000$	25 semi	$3 \cdot 10^6$ FE

$$P_i = \left(1 - \frac{i}{N}\right)^{\alpha \log(\lceil D/M \rceil)}, \qquad \gamma = \beta \frac{D}{M}. \tag{10}$$

Quando l'apprendimento è attivato, ogni coordinata sceglie indipendentemente un dimostratore migliore e aggiorna lo spostamento secondo

$$\Delta x_{i,j}^{t+1} = r_1 \Delta x_{i,j}^{t} + r_2 (x_{k(i,j),j}^{t} - x_{i,j}^{t}) + r_3 \gamma (\bar{x}_j^{t} - x_{i,j}^{t}), \tag{11}$$

con $r_1, r_2, r_3 \sim U(0,1)$ e $\bar{x}^t$ media della popolazione. La popolazione è mantenuta fissa per garantire la parità FE con gli altri metodi; ciò differisce dalla raccomandazione originale $M + \lceil D/10 \rceil$ e viene registrato come adattamento sperimentale. Anche SL-PSO utilizza la mappa comune di gestione dei vincoli.

Tutti e sette i metodi condividono dimensione della popolazione, dominio e budget FE esatto. SL-PSO inizializza lo spostamento a zero, mentre le altre baseline PSO usano la frazione iniziale dichiarata nella configurazione. Tutti gli aggiornamenti sono costruiti da uno snapshot di generazione. Le campagne confermative utilizzano $N = 40$ e budget multipli di N, cosicché il costo di inizializzazione è di N valutazioni e il budget FE è identico per tutti i metodi. I semi sono determinati da una funzione hash stabile dell'identificatore del problema e del seme di campagna. Lo stesso identificatore di seme è associato a ogni metodo per consentire l'appaiamento statistico, ma ciò non implica coordinate iniziali identiche: CPSO C++ e PSO NumPy utilizzano generatori casuali differenti. Le baseline non vengono etichettate come riproduzioni complete degli standard SPSO-2007 o SPSO-2011.

Le campagne complete sono onerose: ciascuna delle due suite COCO comprende 15 120 run metodo-problema; CEC 2013 LSGO comprende 2 625 run e 7.875 miliardi di valutazioni complessive sui sette metodi. Gli smoke test verificano solo l'integrazione, il budget FE, la serializzazione e il resume; non sono utilizzati per classificare gli algoritmi. Nel presente manoscritto non sono ancora riportati risultati ottenuti con le suite esterne.

Disponibilità di dati e codice

Il codice sorgente e le configurazioni sperimentali sono inclusi nella repository di sviluppo associata al manoscritto. La campagna definitiva non è ancora stata eseguita integralmente; pertanto, al momento non sono disponibili dataset completi di risultati grezzi né un archivio persistente associato a questa versione. Eventuali risultati parziali o smoke test non sono utilizzati come evidenza scientifica nel manoscritto.

Riferimenti bibliografici

[1] James Kennedy and Russell Eberhart. Particle swarm optimization. In *Proceedings of ICNN'95 – International Conference on Neural Networks*, volume 4, pages 1942–1948, Perth, Australia, 1995. IEEE. doi: 10.1109/ICNN.1995.488968.

[2] Enrique Alba and Marco Tomassini. Parallelism and evolutionary algorithms. *IEEE Transactions on Evolutionary Computation*, 6(5):443–462, 2002. doi: 10.1109/TEVC.2002.800880.

[3] Erick Cantú-Paz. *Efficient and Accurate Parallel Genetic Algorithms*, volume 1 of *Genetic Algorithms and Evolutionary Computation*. Kluwer Academic Publishers, 2001. ISBN 978-0-7923-7221-9. doi: 10.1007/978-1-4615-4369-5.

[4] Soniya Lalwani, Harish Sharma, Suresh C. Satapathy, Kusum Deep, and Jagdish C. Bansal. A survey on parallel particle swarm optimization algorithms. *Arabian Journal for Science and Engineering*, 44(4):2899–2923, 2019. doi: 10.1007/s13369-018-03713-6.

[5] Calogero Orlando and Angela Ricciardello. Analytic solution of the continuous particle swarm optimization problem. *Optimization Letters*, 15(6):2005–2015, 2021. doi: 10.1007/s11590-020-01671-3.

[6] Maurice Clerc and James Kennedy. The particle swarm – explosion, stability, and convergence in a multidimensional complex space. *IEEE Transactions on Evolutionary Computation*, 6(1):58–73, 2002. doi: 10.1109/4235.985692.

[7] Hassan M. Emara and Hossam A. Abdel Fattah. Continuous swarm optimization technique with stability analysis. In *Proceedings of the 2004 American Control Conference*, pages 2811–2817, Boston, MA, USA, 2004. doi: 10.23919/ACC.2004.1383892.

[8] Hui Huang, Jinniao Qiu, and Konstantin Riedl. On the global convergence of particle swarm optimization methods. *Applied Mathematics & Optimization*, 88(2): Paper No. 30, 2023. doi: 10.1007/s00245-023-09983-3.

[9] James Kennedy and Rui Mendes. Population structure and particle swarm performance. In *Proceedings of the 2002 Congress on Evolutionary Computation (CEC'02)*, volume 2, pages 1671–1676, Honolulu, HI, USA, 2002. IEEE. doi: 10.1109/CEC.2002.1004493.

[10] Nikolaus Hansen, Anne Auger, Raymond Ros, Olaf Mersmann, Tea Tušar, and Dimo Brockhoff. COCO: a platform for comparing continuous optimizers in a black-box setting. *Optimization Methods and Software*, 36(1): 114–144, 2021. doi: 10.1080/10556788.2020.1808977.

[11] Konstantinos Varelas, Ouassim Ait Elhara, Dimo Brockhoff, Nikolaus Hansen, Duc Manh Nguyen, Tea Tušar, and Anne Auger. Benchmarking large-scale continuous optimizers: the bbob-largescale testbed, a COCO software guide and beyond. *Applied Soft Computing*, 97: 106737, 2020. doi: 10.1016/j.asoc.2020.106737.

[12] Xiaodong Li, Ke Tang, Mohammad N. Omidvar, Zhenyu Yang, and Kai Qin. Benchmark functions for the CEC'2013 special session and competition on large-scale global optimization. Technical report, Evolutionary Computation and Machine Learning Group, RMIT University, Melbourne, Australia, 2013.

[13] Yuhui Shi and Russell C. Eberhart. A modified particle swarm optimizer. In *Proceedings of the 1998 IEEE International Conference on Evolutionary Computation*, pages 69–73, Anchorage, AK, USA, 1998. IEEE. doi: 10.1109/ICEC.1998.699146.

[14] Jing J. Liang, A. K. Qin, P. N. Suganthan, and S. Baskar. Comprehensive learning particle swarm optimizer for global optimization of multimodal functions. *IEEE Transactions on Evolutionary Computation*, 10(3):281–295, 2006. doi: 10.1109/TEVC.2005.857610.

[15] Ran Cheng and Yaochu Jin. A social learning particle swarm optimization algorithm for scalable optimization. *Information Sciences*, 291:43–60, 2015. doi: 10.1016/j.ins.2014.08.039.